ThalaSwap V2

Security Assessment

January 29th, 2025 — Prepared by OtterSec

Andreas Mantzoutas andreas@osec.io

Bartłomiej Wierzbiński dark@osec.io

Robert Chen r@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **4**

Vulnerabilities **5**

OS-THL-ADV-00 | Bypassing Funds Repayment via Double Upscaling 6

General Findings **8**

OS-THL-SUG-00 | Missing Validation Logic 9

OS-THL-SUG-01 | Discrepancies in Event Emissions 11

OS-THL-SUG-02 | Code Maturity 13

Appendices

Vulnerability Rating Scale **14**

Procedure **15**

© 2025 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Thala Protocol engaged OtterSec to assess the `thalaswap-v2` and `thalaswap-math-v2` programs. This assessment was conducted between January 16th and January 24th, 2025. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 4 findings throughout this audit engagement.

In particular, we identified a critical loss-of-funds vulnerability concerning the double upscaling of balances in the metastable pool, which may result in incorrect flashloan repayment calculations, allowing the avoidance of repaying borrowed funds (OS-THL-ADV-00).

We also made recommendations to incorporate proper validations (OS-THL-SUG-00) and made suggestions to ensure adherence to coding best practices (OS-THL-SUG-02). We further advised utilizing the proper parameters during emission of events for proper logging of data (OS-THL-SUG-01).

© 2025 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/ThalaLabs/thala-modules/thalaswap_v2. This audit was performed against commit 5700bce.

A brief description of the program is as follows:

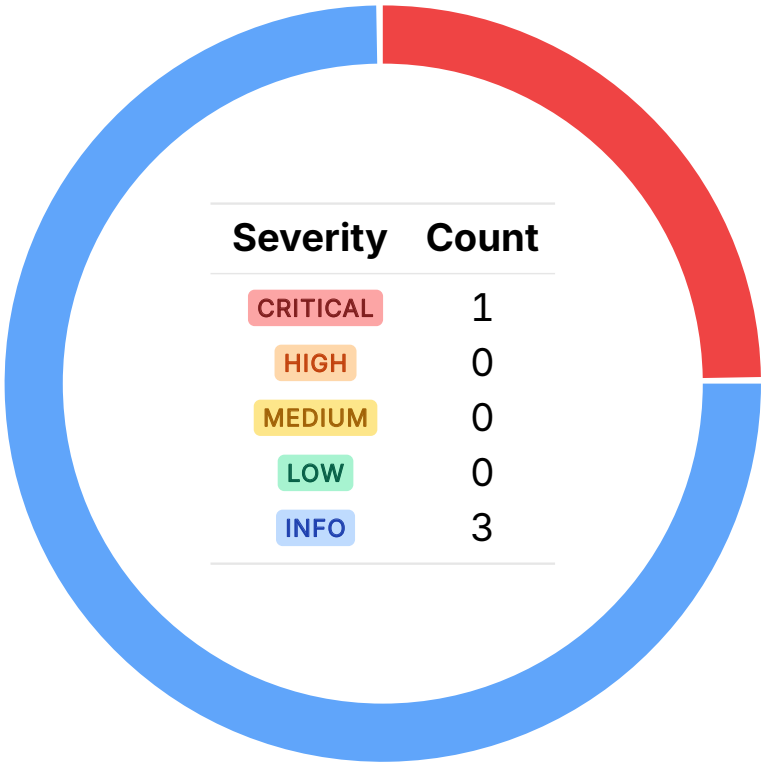
Name	Description
thalaswap-v2	A multi-asset stable pool for stablecoins and weighted pools.
thalaswap-math-v2	IT provides mathematical utilities for two types of liquidity pools in ThalaSwap, weighted pools and stable swap pools.

© 2025 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 4 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	1
HIGH	0
MEDIUM	0
LOW	0
INFO	3

© 2025 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-THL-ADV-00	CRITICAL	RESOLVED	The double upscaling of balances in the metastable pool may result in incorrect flashloan repayment calculations, allowing the avoidance of repaying borrowed funds.

© 2025 Otter Audits LLC. All Rights Reserved.

Bypassing Funds Repayment via Double Upscaling CRITICAL OS-THL-ADV-00

Description

The vulnerability arises from double upscaling during the repayment process in `pay_flashloan` when handling meta-stable pools. Specifically, `pay_flashloan` upscales `balance_after_flashloan` twice. When handling meta-stable pools the funds are multiplied by their value derived from an oracle. As a result the post-repayment invariant computation utilizes an incorrectly scaled value.

>_ *thalaswap_v2/sources/pool.move* RUST

```rust
public fun pay_flashloan(assets: vector<FungibleAsset>, loan: Flashloan) acquires PauseFlag,
    ↪  Pool, MetaStablePool, StablePool, ThalaSwap, WeightedPool {
    [...]
    if (pool_is_metastable(pool_obj)) {
        borrow_amounts = upscale_metastable_amounts(pool_obj, borrow_amounts);
        balances = upscale_metastable_amounts(pool_obj, pool_balances(pool_obj));
    };
    [...]
    while (i < len) {
        let repay_sub_fees = *vector::borrow(&repay_amounts, i) - *vector::borrow(&fee_amounts,
            ↪  i);
        let balance_after_flashloan = *vector::borrow(&balances, i) + repay_sub_fees;
        vector::push_back(&mut balances_after_flashloan, balance_after_flashloan);
        i = i + 1;
    };
    [...]
    if (pool_is_metastable(pool_obj)) {
        balances_after_flashloan = upscale_metastable_amounts(pool_obj,
            ↪  balances_after_flashloan);
    };
    [...]
}
```

Consequently, this creates a discrepancy where the post-repayment invariant (`curr_invariant`) appears larger than the pre-repayment invariant (`prev_invariant`), even if no real repayment has occurred. This incorrect invariant validation allows the flashloan repayment to proceed without properly verifying the adequacy of the repayment. Thus, the pool may accept a repayment that is less than the borrowed amount, resulting in loss of funds

Remediation

Eliminate the double upscaling to prevent inflation of the `curr_invariant`.

© 2025 Otter Audits LLC. All Rights Reserved.

Patch

Fixed in `19dc5f1`.

© 2025 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-THL-SUG-00	There are several instances where proper validation is not done, resulting in potential security issues.
OS-THL-SUG-01	In `AddLiquidityEvent` and `SwapEvent` the incorrect amount values are emitted resulting in faulty event logs.
OS-THL-SUG-02	Suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices.

© 2025 Otter Audits LLC. All Rights Reserved.

Missing Validation Logic

OS-THL-SUG-00

Description

1. There is an absence of validation for asset decimals when creating a metastable pool in `pool::validate_assets_metadata`. Currently, the function only includes a check for the allowed decimal range for assets in Stable pools. This will result in inconsistent handling of token amounts, as scaling or interacting with the assets within the pool may become inaccurate.

>_ *thalaswap_v2/sources/pool.move* RUST

```rust
fun validate_assets_metadata(metadata: vector<Object<Metadata>>, pool_type: u8): bool {
    [...]
    if (pool_type == POOL_TYPE_STABLE) {
        // Check that pool decimals are within bounds
        let j = 0;
        while (j < num_assets) {
            let x = vector::borrow(&metadata, j);
            let decimals = fungible_asset::decimals(*x);
            if (decimals > MAX_STABLE_DECIMALS_SUPPORTED ||
                ↪  decimals<MIN_STABLE_DECIMALS_SUPPORTED) return false;
            j = j + 1;
        };
    };
    [...]
}
```

2. The assert condition in `pools::flashloan` checks that the specified amount for each asset in the flashloan does not exceed the available pool balance. However, it utilizes the `<=` condition which permits borrowing the entire balance of an asset in the pool. This may result in a temporary drainage of the entire pool's liquidity for the duration of the flashloan.

>_ *thalaswap_v2/sources/pool.move* RUST

```rust
  public fun flashloan(pool_obj: Object<Pool>, amounts: vector<u64>):
      ↪  (vector<FungibleAsset>, Flashloan) acquires PauseFlag, Pool {
    [...]
    vector::zip(pool.assets_metadata, amounts, |metadata, amount| assert!(amount <=
        ↪  primary_fungible_store::balance(pool_addr, metadata),
        ↪  ERR_POOL_FLASHLOAN_INVALID_AMOUNT));
    [...]
}
```

© 2025 Otter Audits LLC. All Rights Reserved.

Remediation

1. Ensure to validate the decimal range for metastable pools as well in `validate_assets_metadata`.
2. Modify the assert condition to strictly check that the flashloan amount is less than the pool balance (`<` instead of `<=`).

Patch

1. Fixed in 19dc5f1.
2. Fixed in 7df3ae8.

© 2025 Otter Audits LLC. All Rights Reserved.

Discrepancies in Event Emissions OS-THL-SUG-01

Description

1. In the current implementation of `pool::add_liquidity_weighted`, `AddLiquidityEvent` emits `amounts` (the raw values of assets initially provided to the function) instead of `remained_assets` which is the actual amounts of assets the user deposited into the pool.

>_ thalaswap_v2/sources/pool.move RUST

```rust
public fun add_liquidity_weighted(pool_obj: Object<Pool>, assets: vector<FungibleAsset>):
    ↪ (FungibleAsset, vector<FungibleAsset>) acquires PauseFlag, Pool {
    [...]
    event::emit(AddLiquidityEvent {
        pool_obj,
        metadata,
        amounts,
        minted_lp_token_amount: preview.minted_lp_token_amount,
        pool_balances: pool_balances(pool_obj),
    });
    (lp_token, refunds)
}
```

2. In the current implementations of `swap_exact_out_stable` and `swap_exact_out_metastable`, the `amount_in` value logged by the `SwapEvent` is provided by the users. However, the `amount_in` that is passed by the user may differ from the actual `amount_in` needed to achieve the desired output (`amount_out`).

>_ thalaswap_v2/sources/pool.move RUST

```rust
public fun swap_exact_out_stable([...]) [...] {
    [...]
    event::emit(SwapEvent {
        pool_obj,
        metadata: pool_assets_metadata(pool_obj),
        idx_in: preview.idx_in,
        idx_out: preview.idx_out,
        amount_in,
        amount_out,
        total_fee_amount: preview.total_fee_amount,
        protocol_fee_amount: preview.protocol_fee_amount,
        pool_balances: pool_balances(pool_obj),
    });
    [...]
}
```

© 2025 Otter Audits LLC. All Rights Reserved.

Remediation

1. Replace `amounts` in `AddLiquidityEvent` with `remained_assets` since it represents the actual assets deposited into the pool.
2. Pass the exact input amount that was actually utilized in the swap (`preview.amount_in`) to `SwapEvent` to ensure proper logging of swap parameters.

Patch

1. Fixed in 7df3ae8.
2. Fixed in 19dc5f1.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Maturity

OS-THL-SUG-02

Description

1. If the flashloan fee is lower than the swap fee, users may exploit this economic imbalance by repeatedly using flashloans for swaps, effectively bypassing swap fees to achieve the same outcome at a lower cost. This undermines the protocol's revenue and reduces rewards for liquidity providers. To prevent this, the flashloan fee should be set equal to or higher than the swap fee.
2. `weighted_pool_tests` focuses on a specific, simple ratio/asset set utilizing a `50:50` weight and 2 assets, failing to reflect the diversity of real-world scenarios that the application may encounter. As a result of this narrow scope, complex conditions and edge cases remain untested.

Remediation

Implement the above-mentioned suggestions.

Patch

Fixed in 19dc5f1.

© 2025 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2025 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2025 Otter Audits LLC. All Rights Reserved.